

Practical 6

```
In [131... import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```
In [132... df=pd.read_csv("Iris.csv")
```

```
In [133... df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Id                     150 non-null   int64
1   SepalLengthCm         150 non-null   float64
2   SepalWidthCm          150 non-null   float64
3   PetalLengthCm         150 non-null   float64
4   PetalWidthCm          150 non-null   float64
5   Species               150 non-null   str
dtypes: float64(4), int64(1), str(1)
memory usage: 7.2 KB
```

```
In [134... df.isnull().sum()
```

```
Out[134... Id                     0
SepalLengthCm             0
SepalWidthCm              0
PetalLengthCm             0
PetalWidthCm              0
Species                   0
dtype: int64
```

```
In [135... df.describe()
```

```
Out[135...      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
count  150.000000      150.000000      150.000000      150.000000      150.000000
mean    75.500000         5.843333         3.054000         3.758667         1.198667
std     43.445368         0.828066         0.433594         1.764420         0.763161
min      1.000000         4.300000         2.000000         1.000000         0.100000
25%     38.250000         5.100000         2.800000         1.600000         0.300000
50%     75.500000         5.800000         3.000000         4.350000         1.300000
75%    112.750000         6.400000         3.300000         5.100000         1.800000
max    150.000000         7.900000         4.400000         6.900000         2.500000
```

```
In [136... from sklearn import preprocessing
```

```
In [137... #one_hot=preprocessing.OneHotEncoder()
#encoded=one_hot.fit_transform(df[['Species']]).toarray()
```

```
In [138... from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
df['Species']=le.fit_transform(df['Species'])
#encoded
df
```

Out [138...

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	0
1	2	4.9	3.0	1.4	0.2	0
2	3	4.7	3.2	1.3	0.2	0
3	4	4.6	3.1	1.5	0.2	0
4	5	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	2
146	147	6.3	2.5	5.0	1.9	2
147	148	6.5	3.0	5.2	2.0	2
148	149	6.2	3.4	5.4	2.3	2
149	150	5.9	3.0	5.1	1.8	2

150 rows × 6 columns

In [139...

```
#independent variable
x=df.drop("Species",axis=1)
#dependent variable
y=df["Species"]
```

In [140...

```
print(x.head())
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
0	1	5.1	3.5	1.4	0.2
1	2	4.9	3.0	1.4	0.2
2	3	4.7	3.2	1.3	0.2
3	4	4.6	3.1	1.5	0.2
4	5	5.0	3.6	1.4	0.2

In [141...

```
print(y.head())
```

```
0    0
1    0
2    0
3    0
4    0
Name: Species, dtype: int64
```

In [142...

```
x_train,x_test,y_train,y_test=train_test_split(x,y, test_size=0.2)
model=GaussianNB()
model.fit(x_train,y_train)
y_pred=model.predict(x_test)
```

In [143...

```
print("Predictions:", y_pred)
```

```
Predictions: [2 1 1 1 1 2 2 0 0 2 0 1 2 0 1 0 0 1 1 2 2 1 2 2 2 2 0 1 0]
```

In [144...

```
#cm=confusion_matrix(y_test,y_pred)
#cm
```

In [145...

```
#print("TP= ",cm[1][1])
#print("FP= ",cm[0][1])
#print("TN= ",cm[0][0])
#print("FN= ",cm[1][0])
```

In [146...

```
Accuracy=accuracy_score(y_test,y_pred)
print("Accuracy= ",Accuracy)
```

```
Accuracy= 1.0
```

In [147...

```
Precision=precision_score(y_test,y_pred,average='micro')
```

```
print("Precision= ",Precision)
```

Precision= 1.0

```
In [148... Recall=recall_score(y_test,y_pred,average='micro')
print("Recall= ",Recall)
```

Recall= 1.0

```
In [149... F1=f1_score(y_test,y_pred,average='micro')
print("F1= ",F1)
```

F1= 1.0

```
In [150... cm=confusion_matrix(y_test,y_pred)
cm
```

```
Out[150... array([[ 8,  0,  0],
          [ 0, 10,  0],
          [ 0,  0, 12]])
```

```
In [151... for i in range(len(cm)):
    TP = cm[i][i]
    FP = sum(cm[:, i]) - TP
    FN = sum(cm[i, :]) - TP
    TN = cm.sum() - (TP + FP + FN)

    print(f"Class {i}: TP={TP}, FP={FP}, FN={FN}, TN={TN}")
```

Class 0: TP=8, FP=0, FN=0, TN=22
Class 1: TP=10, FP=0, FN=0, TN=20
Class 2: TP=12, FP=0, FN=0, TN=18

In []:

In []: